

Capítulo 2

****Nome**:** Davi dos Santos Mattos
119133049

****DRE**:**

P4 -- HTTP GET - parsing humano

- a) Qual é o URL do documento requisitado pelo navegador?
 - a. Resposta: `[http://gaia.cs.umass.edu/cs453/index.html]`
`(http://gaia.cs.umass.edu/cs453/index.html)`.
- b) Qual versão do HTTP o navegador está rodando?
 - a. Resposta: `HTTP/1.1`.
- c) O navegador requisita uma conexão não persistente ou persistente?
 - a. O cabeçalho `Connection: keep-alive` indica reutilização do socket.
 - b. Resposta: Persistente (`keep-alive`).
- d) Qual é o endereço IP do hospedeiro no qual o navegador está rodando?
 - a. Resposta: Não é possível determinar. O endereço IP do cliente pertence aos cabeçalhos do protocolo IP (camada de rede), não estando presente no payload da mensagem HTTP (camada de aplicação).
- e) Qual tipo de navegador inicia essa mensagem? Por que o tipo de navegador é necessário?
 - a. Resposta: É o `Mozilla/5.0` (Netscape 7.2 no Windows NT 5.1). O servidor usa essa informação para negociar conteúdo e enviar versões formatadas especificamente para aquele navegador.

P7 -- web browser

1. **Etapa DNS:** Para traduzir o nome de domínio para endereço IP visitando n servidores DNS, o tempo decorrido é a soma das latências individuais: $\sum_{i=1}^n RTT_i$.
2. **Etapa TCP Handshake:** Aberta a conexão, consome-se $1 \cdot RTT_0$ para sincronizar os pacotes SYN e SYN/ACK.
3. **Etapa HTTP Request/Response:** O envio da requisição GET e a recepção do texto HTML consomem mais $1 \cdot RTT_0$ (desprezando o tempo de transmissão físico por ser muito pequeno).

$$\text{Tempo Total} = \left(\sum_{i=1}^n RTT_i \right) + 2 \cdot RTT_0$$

P8 -- web browser

a) HTTP Não Persistente sem Conexões TCP Paralelas

- Cada objeto exige 1 RTT_0 para abrir o TCP e 1 RTT_0 para buscar o arquivo.
- Para o arquivo base + 8 objetos (9 objetos no total):

$$T = RTT_{\text{DNS}} + 2 \cdot RTT_0 + 8 \times (2 \cdot RTT_0) = RTT_{\text{DNS}} + 18 \cdot RTT_0$$

b) HTTP Não Persistente com 6 Conexões Paralelas

- **HTML Base:** $RTT_{\text{DNS}} + 2 \cdot RTT_0$.
- **8 Objetos em paralelo (Limite de 6 por vez):**
 - a. Lote 1 (6 objetos em paralelo): $2 \cdot RTT_0$.
 - b. Lote 2 (2 objetos restantes em paralelo): $2 \cdot RTT_0$.

$$T = RTT_{\text{DNS}} + 2 \cdot RTT_0 + 2 \cdot RTT_0 + 2 \cdot RTT_0 = RTT_{\text{DNS}} + 6 \cdot RTT_0$$

c) HTTP Persistente (com Pipelining)

- Conexão aberta no HTML base ($2 \cdot RTT_0$).
- Os 8 objetos são pedidos juntos numa única rajada e respondidos consecutivamente ($1 \cdot RTT_0$).

$$T = RTT_{\text{DNS}} + 2 \cdot RTT_0 + 1 \cdot RTT_0 = RTT_{\text{DNS}} + 3 \cdot RTT_0$$

P9 -- caching -- interessante!

a) Sem Cache Instalação

1. Taxa de Chegada de Tráfego (β):

$$\beta = 16 \text{ req/s} \times 1 \text{ Mbit} = 16 \text{ Mbps}$$

2. Utilização do Enlace (I):

$$I = \frac{16 \text{ Mbps}}{15 \text{ Mbps}} = 1,067$$

3. **Análise:** Como $I > 1$, a fila cresce de forma ilimitada, o atraso de acesso $d_{\text{acesso}} \rightarrow \infty$, tornando o tempo de resposta total infinito (falha na rede).

b) Com Cache Instalado na LAN ($h = 0,4$)

1. **Novo tráfego no enlace de acesso:** Apenas a fração não satisfeita localmente ($1 - h = 0,6$) passa pelo enlace.

$$\beta_{\text{novo}} = (1 - 0,4) \times 16 \text{ Mbps} = 9,6 \text{ Mbps}$$

2. **Nova Intensidade de Tráfego (I_{novo}):**

$$I_{\text{novo}} = \frac{9,6 \text{ Mbps}}{15 \text{ Mbps}} = 0,64$$

3. **Calculando Δ :**

$$\Delta = \frac{1 \text{ Mbit}}{15 \text{ Mbps}} = 0,0667 \text{ s}$$

4. **Calculando d_{acesso} :**

$$d_{\text{acesso}} = \frac{0,0667}{1 - 0,64} = \frac{0,0667}{0,36} \approx 0,185 \text{ s}$$

5. **Calculando o Tempo Médio de Resposta Total ($T_{\text{médio}}$):** (Considerando $d_{\text{LAN}} \approx 0 \text{ s}$)

$$T_{\text{médio}} = 0,4 \cdot (0) + 0,6 \cdot (0,185 + 3) = 0,6 \cdot (3,185) \approx 1,911 \text{ segundos}$$

P10 -- HTTP persistente x não persistente

Cenário:

- Enlace de 10 metros com taxa 150 bits/s em ambas as direções.
- Pacote de dados = 100.000 bits (100 Kbits).
- Pacote de controle (ACK/Handshake) = 200 bits.
- Página inicial contém 10 objetos referenciados de 100 Kbits.

Análise para Instâncias Paralelas Não Persistentes:

- **Dividindo a Banda:** Com N conexões paralelas, cada conexão recebe uma fatia igual $\frac{150}{N}$ bits/s.
- **Conclusão:** Abrir N conexões não persistentes **não traz benefícios reais de tempo de transmissão**, pois a largura de banda total do enlace é fixa (150 bits/s). Na verdade, piora a eficiência total devido ao *overhead* repetido de pacotes de controle de 200 bits multiplicados por cada handshake.

Análise para HTTP Persistente:

- **Desempenho:** O HTTP persistente evita 10 handshakes TCP e economiza pacotes de controle extras no enlace de baixa largura de banda.
- **Ganhos: Sim, os ganhos são significativos**, eliminando a latência de controle e a divisão desnecessária de banda em conexões paralelas concorrentes.

P13 -- calcule também o tempo médio para a transmissão de objetos em cada caso - qual o tempo médio de transmissão de objetos no item a)? qual o tempo médio de transmissão de objetos no item b)? explique porque o tempo médio depende da ordem de serviço

- **Vídeo:** 2.000 quadros
- **5 imagens:** 3 quadros cada = 15 quadros no total
- **Total de quadros a enviar:** $2.000 + 15 = 2.015$

a) Sem interleaving (vídeo inteiro primeiro)

O vídeo ocupa os 2.000 primeiros "tempos de quadro"; só depois disso as 5 imagens são enviadas, uma após a outra:

$$2.000 + (5 \times 3) = 2.000 + 15 = \boxed{2.015 \text{ tempos de quadro}}$$

(Esse é o tempo até a **última** imagem terminar.)

b) Com interleaving

Com HTTP/2, os quadros do vídeo e das 5 imagens são intercalados — o servidor envia 1 quadro de cada "stream" por vez, em rodízio (round-robin): vídeo, img1, img2, img3, img4, img5, vídeo, img1, ...

Cada **rodada** consome 6 tempos de quadro (1 de cada stream). Como cada imagem só precisa de **3 quadros**, ela termina após **3 rodadas**:

$$3 \text{ rodadas} \times 6 \text{ streams} = \boxed{18 \text{ tempos de quadro}}$$

(A última imagem — a 5ª — recebe seu quadro final exatamente no tempo de quadro 18.)

c) Tempo médio de transmissão de objetos — item a) (sem interleaving)

Calculando o instante em que **cada um dos 6 objetos** (vídeo + 5 imagens) termina de ser recebido:

Objeto	Termina em (tempo de quadro)
Vídeo	2.000
Imagem 1	2.003
Imagem 2	2.006
Imagem 3	2.009

Objeto	Termina em (tempo de quadro)
Imagem 4	2.012
Imagem 5	2.015

$$\text{Tempo médio} = \frac{2000 + 2003 + 2006 + 2009 + 2012 + 2015}{6} = \frac{12045}{6} = \boxed{2.007,5 \text{ tempos de quadro}}$$

d) Tempo médio de transmissão de objetos — item b) (com interleaving)

Objeto	Termina em (tempo de quadro)
Imagem 1	14
Imagem 2	15
Imagem 3	16
Imagem 4	17
Imagem 5	18
Vídeo	2.015

$$\text{Tempo médio} = \frac{14 + 15 + 16 + 17 + 18 + 2015}{6} = \frac{2095}{6} \approx \boxed{349,17 \text{ tempos de quadro}}$$

e) Por que o tempo médio depende da ordem de serviço

O **makespan** (tempo até *tudo* terminar) é **idêntico** nos dois casos — 2.015 tempos de quadro, já que o total de trabalho (quadros) enviado é o mesmo, não importa a ordem. Mas o **tempo médio de transmissão dos objetos individuais** despenca de ~2.007 para ~349 com o interleaving.

Isso acontece porque, sem interleaving, **todas as 5 imagens pequenas ficam "presas" atrás do vídeo gigante**, mesmo sendo objetos pequenos e rápidos de enviar, elas só começam depois que os 2.000 quadros do vídeo terminam. Com interleaving, as imagens **"furam a fila"** e terminam quase imediatamente (por volta do quadro 14-18), enquanto só o vídeo (que é grande e o usuário já espera demorar) fica para o final.

Isso é exatamente o mesmo princípio de **escalonamento "menor trabalho primeiro" (Shortest Job First)** que minimiza o **tempo médio de espera** em filas — mesmo que o tempo total do sistema não mude, atender primeiro os itens pequenos reduz drasticamente a **média**, porque poucos itens (o vídeo) "puxam" a média pra cima quando são atendidos por último, ao invés de "travarem" todos os outros itens atrás deles.

P23 -- esse é muito importante -- fala sobre escalonamento p2p

a) Se $u_s/N \leq d_{min}$

Para $t = NF/u_s$. Isso vale quando o **servidor é o gargalo**, ou seja, mesmo dividindo a capacidade do servidor igualmente entre os N peers (u_s/N para cada um), essa fatia ainda é **menor ou igual** à

capacidade de download de qualquer peer (d_{min}). Nesse caso, os peers **nunca** ficam esperando por falta de capacidade de download, o gargalo é inteiramente do lado do servidor.

Esquema: o servidor envia o arquivo sequencialmente (ou dividido) a cada peer, usando toda sua capacidade u_s dividida entre os N . O tempo total é NF/u_s .

b) Se $u_s/N \geq d_{min}$

Aqui a situação se inverte: o servidor **tem capacidade de sobra**, dividindo u_s entre os N peers, cada fatia (u_s/N) seria **maior** do que a capacidade de download do peer mais lento (d_{min}). Nesse caso, o **gargalo passa a ser o peer mais lento**, não mais o servidor.

Esquema: o servidor pode enviar para cada peer na **taxa máxima que aquele peer consegue receber**. O peer mais lento, com capacidade d_{min} , vai levar:

$$t = \frac{F}{d_{min}}$$

c) Conclusão — fórmula geral

Combinando os dois casos, o tempo mínimo de distribuição, na arquitetura cliente-servidor, é:

$$t_{cliente-servidor} \geq \max \left(\frac{NF}{u_s}, \frac{F}{d_{min}} \right)$$

O sistema é sempre limitado pelo **maior** dos dois "gargalos possíveis". Nenhum esquema de distribuição consegue ser mais rápido que esse limite.

P24 -- também interessante sobre p2p

- F : Tamanho total do arquivo a ser distribuído, medido em **bits**.
- N : Número de pares (clientes) que desejam receber o arquivo.
- u_s : Taxa máxima de *upload* do enlace de acesso do servidor, em **bits/s**
- u_i : Taxa máxima de *upload* do i -ésimo par, em **bits/s**.
- $\sum_{i=1}^N u_i$: Capacidade total de *upload* agregada fornecida por todos os N pares, em **bits/s**.
- d_{min} : Menor taxa de *download* entre todos os clientes (assumida muito grande e não limitante no exercício).
- D_{P2P} : Tempo total de distribuição no modelo P2P, em **segundos**.

a) Suponha que $u_s \leq \frac{u_s + u_1 + \dots + u_N}{N}$. Especifique um esquema de distribuição que possua tempo de distribuição de $\frac{F}{u_s}$.

- **Esquema de Distribuição:**

1. O servidor transmite o arquivo continuamente para a rede a sua taxa máxima u_s .

a. Como a taxa média agregada de upload por par $\frac{u_s + \sum u_i}{N}$ é **maior ou igual** à capacidade do servidor u_s , os pares juntos possuem capacidade de upload mais do que suficiente para replicar internamente qualquer fluxo que chegue do servidor.

b. O servidor divide a transmissão de u_s enviando fatias para diferentes pares. Cada par, assim que recebe um bit do servidor, o redistribui para os demais pares utilizando sua própria capacidade de upload.

- **Cálculo do Tempo de Distribuição:** Como a capacidade de redistribuição da rede de pares não é o gargalo, o fator limitante do sistema é simplesmente o tempo necessário para o servidor injetar **uma única cópia completa** do arquivo de F bits na rede:

$$D_{P2P} = \frac{F}{u_s}$$

b) Suponha que $u_s > \frac{u_s + u_1 + \dots + u_N}{N}$. Especifique um esquema de distribuição que possua tempo de distribuição de $\frac{NF}{u_s + u_1 + \dots + u_N}$.

- **Esquema de Distribuição:**

a. Neste cenário, a capacidade de upload do servidor u_s é superior à média de upload por par da comunidade. O gargalo do sistema passa a ser a **capacidade de upload total agregada do sistema** ($u_{\text{total}} = u_s + \sum u_i$).

b. O servidor e todos os N pares operam a **100% de suas capacidades de upload** continuamente durante todo o processo.

c. O servidor envia blocos inéditos aos pares e os pares redistribuem esses blocos entre si de forma perfeitamente balanceada.

- **Cálculo do Tempo de Distribuição:** Para entregar 1 cópia do arquivo de F bits para N pares, o sistema como um todo precisa realizar o upload total de $N \cdot F$ bits de dados. Com o sistema transmitindo à taxa agregada máxima de $u_s + \sum_{i=1}^N u_i$, o tempo total necessário para concluir o upload dos $N \cdot F$ bits é:

$$D_{P2P} = \frac{NF}{u_s + \sum_{i=1}^N u_i}$$

c) Conclua que o tempo mínimo de distribuição é, em geral, dado por

$$\max \left\{ \frac{F}{u_s}, \frac{NF}{u_s + u_1 + \dots + u_N} \right\}.$$

- **Demonstração/Conclusão:**

1. **Restrição do Servidor:** Para que a comunidade de pares receba o arquivo, pelo menos 1 cópia completa de F bits precisa sair do servidor. Como a taxa máxima de upload do servidor é u_s , temos obrigatoriamente $D_{P2P} \geq \frac{F}{u_s}$.

b. **Restrição da Capacidade Agregada:** O trabalho total de upload exigido pela aplicação é $N \cdot F$ bits. A taxa máxima com que a rede inteira consegue realizar uploads é $u_s + \sum u_i$. Logo, temos $D_{P2P} \geq \frac{NF}{u_s + \sum u_i}$.

c. **Restrição de Download (Geral):** Se $d_{\min}$ não fosse infinito, teríamos também a restrição $D_{P2P} \geq \frac{F}{d_{\min}}$.

Unindo as restrições físicas (e considerando $d_{\min}$ muito grande), o tempo mínimo de distribuição em P2P é o limite inferior mais rigoroso (o valor máximo entre as restrições):

$$D_{P2P} = \max \left\{ \frac{F}{u_s}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right\}$$

Site que usa cookies:

Escolha um site que você goste e tente descobrir como que ele usa cookies -- para isso, estude o HTML do site. Indique também quais outros sites são acessados, sem você saber, quando você acessa o site em questão

Site: <https://ge.globo.com/futebol/times/fluminense/>

- Categoria 1 — Google Analytics / Google Ads (terceiros)

`_ga`, `_ga_3EWSSPPX...`, `_ga_5401XJ0K8J`, `_ga_K886Y0T4C5`, `_ga_NWTRTWY...`,
`_ga_P4F3TC8HVE`, `_ga_SL5WEXQ2...`, `_ga_WLHSK1RZ...`, `_ga_YEWKNJM...`, `_gcl_au`

Esse é, de longe, o grupo mais numeroso. O padrão `_ga_XXXXXXXXXX` (com um código diferente em cada um) é característico do **Google Analytics 4** — cada `_ga_<ID>` corresponde a uma **propriedade de medição específica** dentro do Analytics. O fato de existirem **oito** cookies `_ga_*` diferentes sugere que a Globo mede esse site em **múltiplas propriedades/contas** do Analytics simultaneamente (possivelmente uma pra audiência geral, outra pra publicidade, outra pra um produto específico, etc.). Já o `_gcl_au` é do **Google Ads** — usado para medir conversões de anúncios.

- Categoria 2 — Anti-bot / anti-fraude (terceiro!) — conecta com nossa tentativa de fetch

`datadome`

Datadome é um serviço de terceiro especializado em **detectar e bloquear bots/scrapers** — ele analisa padrões de comportamento (velocidade de cliques, user-agent, etc.) pra diferenciar humanos de programas automatizados.

- Categoria 3 — Identidade/login unificado da Globo (primeiro nível — grupo Globo)

`GBID`, `GLOBO_ID`, `glb_uid`, `glb_uid_jwt`, `gpixel_uid`, `hsid`, `compass_uid`

Esses cookies **não são de terceiros** no sentido estrito (são da própria Globo), mas repare que vários têm domínio começando com `.gl...` (provavelmente `.globo.com`, não `.ge.globo.com`) — ou seja, são cookies configurados para o **domínio "pai"**, compartilhados entre **todos os sites do grupo**

Globo (g1, ge, gshow, globoplay, etc.). É assim que, se você estiver logado em um site da Globo, os outros "já sabem quem você é" sem precisar logar de novo — um sistema de **identidade unificada**. O `glb_uid_jwt` guarda um **JWT** (JSON Web Token — um formato de token de autenticação assinado digitalmente).

- Categoria 4 — Tag Management (terceiro) — orquestra outros terceiros

`utag_main`

Esse é da **Tealium**, uma plataforma de **gerenciamento de tags** (parecida em função com o Google Tag Manager que vimos no HTML) — ela funciona como um "hub" que **dispara outros scripts de rastreamento** de forma centralizada. Isso significa que provavelmente existem **ainda mais** terceiros sendo carregados **indiretamente**, através da Tealium, que não aparecem diretamente como cookies na sua lista.

- Categoria 5 — Preferências/consentimento e personalização (primeira parte, Globo)

`cookie-banner-...`, `w3sthem`, `_pc_cookie_atual`, `_pc_randomCo...`, `_pcid`, `_pctx`, `_pat`, `_tbc`, `xbc`, `hzcc`, `utag_main`

O `cookie-banner-...` guarda se você **já aceitou** o banner de cookies (evita mostrar de novo). `_pc_*` e `_pctx` parecem ligados a algum sistema de **personalização de conteúdo**.

Os principais **domínios de terceiros** carregados ao visitar

`ge.globo.com/futebol/times/fluminense/` são:

Domínio	Empresa/Serviço	Função
<code>google-analytics.com</code> / <code>googletagmanager.com</code>	Google	Analytics e gerenciamento de tags
<code>googlesyndication.com</code> / conversão de <code>_gcl_au</code>	Google Ads	Rastreamento de anúncios
<code>datadome.co</code>	DataDome	Anti-bot/anti-fraude
<code>tealium...</code> (via <code>utag_main</code>)	Tealium	Orquestração de tags — provavelmente dispara ainda mais terceiros
<code>sentry.globoi.com</code>	Sentry (via Globo)	Monitoramento de erros JavaScript
<code>s3.glbimg.com</code>	Amazon S3 (CDN da Globo)	Hospedagem de scripts/imagens do grupo

Uma página de notícias de futebol pode carregar **dezenas de cookies** e se comunica com **pelo menos 4-5 empresas terceiras diferentes** (Google, DataDome, Tealium, Sentry, além da própria

infraestrutura AWS da Globo). Sendo a maior parte disso **invisível** pro usuário comum, cumprindo funções que vão de analytics e publicidade até segurança (anti-bot) e monitoramento técnico.